

Research Report: ROS2 Basics and Architecture

Full Name: **Nguyễn Minh Quang**

Topic: ROS2 Basics Research

April 8, 2026

Contents

1	Introduction: What is ROS2?	3
2	ROS1 vs. ROS2: Key Differences	3
3	Nodes, Topics, Messages	3
3.1	Nodes	3
3.2	Topics	4
3.3	Messages	4
4	Communication Patterns	4
4.1	Publisher/Subscriber	4
4.2	Services	5
4.3	Actions	5
5	Example Flow and Diagram	5
5.1	Flow: Sensor → Processing → Control	5
5.2	System Communication Analysis	6
6	References	7

1 Introduction: What is ROS2?

Robot Operating System (ROS) is not an actual operating system like Windows. Instead, it is a middleware — a set of tools and libraries that act as a translator to help different parts of a robot (sensors, motors, and software) talk to each other easily. The first version, ROS1, was great for learning and research. However, it had some big weaknesses when used in the real world. For example, it relied on a single "Master" node to run everything. If this Master crashed, the entire robot stopped working. It also lacked security and didn't work well over unstable Wi-Fi. ROS2 Humble was specifically designed to overcome the major limitations of its predecessor.

2 ROS1 vs. ROS2: Key Differences

The most significant improvement is its decentralized architecture, which completely removes the "Master Node." This means each part of the robot can now discover and talk to others directly, making the system much more reliable because there is no single point of failure. Additionally, ROS2 adopts the DDS standard to manage data, which ensures stable communication even over unreliable wireless networks. Most importantly, ROS2 supports real-time processing and built-in security encryption. This ensures that critical commands, such as an "Emergency Stop," are executed instantly and safely from outside hacking. In short, while ROS1 was a great start for research, ROS2 Humble provides the professional and robust foundation needed for modern autonomous systems.

Table 1: Comparison between ROS1 and ROS2

Feature	ROS1	ROS2
Architecture	Centralized (ROS Master)	Decentralized (DDS-based)
Real-time Support	Limited	Supported
Platforms	Mainly Ubuntu Linux	Linux, Windows, macOS, RTOS
Security	Minimal native security	Integrated TLS/DDS-Security
Connectivity	High-bandwidth required	Resilient to lossy networks

3 Nodes, Topics, Messages

3.1 Nodes

A Node is a specialized process responsible for a specific task in a robotic system. By decomposing a complex robot "brain" into many small modular pieces, ROS2 ensures that a failure in one area does not necessarily crash the entire system. In a practical scenario like a forklift, there is a laser Node to detect racks and obstacles, a vision Node

to identify barcodes on pallets for precise positioning, a navigation Node to calculate the optimal path from the current location to the target, a motor control Node to move the wheel or lift the fork.

3.2 Topics

Topics are the primary communication route in ROS2, utilizing a Publish/Subscribe model. This system is "anonymous" because a Node sending data (Publisher) does not need to know which Nodes are receiving it (Subscribers). This many-to-many relationship allows multiple sensors to transcend data to various processing Nodes at once. Using the forklift scenario, the LiDAR Node publishes distance data to the `/scan` Topic so other Nodes can monitor surroundings, the vision Node publishes the coordinates of the pallet to the `/pallet_pose` Topic, the navigation Node sends velocity commands to move the vehicle through the `/cmd_vel` Topic.

3.3 Messages

If a Topic is the "road", then a Message is the specific "vehicle" carrying data on that "road". Every topic must have a strictly defined Message Type to ensure all nodes "speak the same language". ROS2 utilizes standardized interfaces, such as `geometry_msgs/msg/Twist`, which defines velocity in 3D space using linear and angular vectors (e.g., `geometry_msgs/msg/Twist` "linear: x: 2.0, y: 1.2, z: 0.0, angular: x: 0.0, y: 3.6, z: 1.8"). Using these consistent data structures, developers can easily swap out different hardware components or algorithms as long as they use the same message format, greatly simplifying the integration of sensors like LiDAR or IMUs.

4 Communication Patterns

ROS2 provides three primary patterns for Node interaction:

4.1 Publisher/Subscriber

The Publisher/Subscriber (Pub/Sub) model works much like a radio broadcast. In a robotic system, certain nodes act as Publishers that constantly send data (Messages) onto a specific channel called a Topic. The advantage of this system is that the Publisher doesn't need to know who is receiving the data; it just sends the signal out. On the other side, any node that wants that data simply subscribes to that Topic.

For a forklift, the LiDAR sensor is a Publisher that constantly broadcasts distance data to the `/scan` topic. Other nodes, like the "Brain" or a safety monitor, just receto this stream to build a map or detect obstacles using that data. Because it is a "Many-

to-Many" system, you can add new features—like a dashboard for the driver—just by having it subscribe to the existing topic without ever changing the forklift's original code.

4.2 Services

Unlike the continuous data flow of Topics, a Service is more like a phone call or a quick text message (Request/Response). This is a two-way communication used for fast, one-time tasks that need an immediate answer. A Client node sends a request to a Server node and typically waits for a reply before continuing its process. This mechanism is ideal for state-based commands or system configurations where confirmation is required.

For a forklift, Services are often used to trigger emergency stops or request battery status updates. For instance, when a power management node needs to know the exact battery level, it sends a Request to the hardware server; the server immediately responds with the current battery percentage and closes the communication session. This ensures that the command was actually received and handled correctly.

4.3 Actions

Actions are the most advanced way nodes talk to each other, designed for tasks that take a long time to finish. Take instance of an Action like ordering food on a delivery app. It consists of three parts: the Goal (the delivery address), the Feedback (where the driver is right now), and the Result (the food has arrived).

In a forklift scenario, a command like "Drive to Shelf A" is a classic Action. While the forklift is moving (which might take 30 seconds), it constantly sends back Feedback to the main computer, such as "Reached 10 meters" or "Turning left." The biggest advantage of an Action is that you can Cancel it at any time. If someone walks in front of the forklift, the system can kill the task immediately instead of waiting for the forklift to reach the shelf.

5 Example Flow and Diagram

5.1 Flow: Sensor → Processing → Control

To illustrate how these concepts work together in a real-world scenario, consider an Autonomous Forklift navigating a warehouse. This system operates through a continuous data pipeline: Sensor → Processing → Control.

First, the Sensor Phase begins with a LiDAR Node. This node acts as a Publisher; it constantly scans the surroundings for obstacles and packages this raw data into a `sensor_msgs/msg/LaserScan` message. It then broadcasts this information onto a specific Topic, typically named `/scan`. The LiDAR node does not need to know which part of

the robot is using the data; it simply ensures the "eyes" of the forklift are providing a constant stream of information.

Next, in the Processing Phase, a Navigation Node acts as a Subscriber to the `/scan` topic. It receives the incoming laser data and performs logic calculations. For example, if the node detects an obstacle within a 0.5-meter threshold, it calculates a maneuver to steer away. Once the decision is made, the Navigation Node switches roles to become a Publisher, sending high-level movement commands—such as "slow down" or "turn left"—onto the `/cmd_vel` (Command Velocity) topic using a `geometry_msgs/msg/Twist` message.

Finally, in the Control Phase, a Motor Driver Node subscribes to the `/cmd_vel` topic. It reads the velocity values (linear and angular) and translates them into electrical signals (such as PWM) for the actual wheel motors. By using this modular approach, the forklift's navigation Node is completely decoupled from the hardware. If the warehouse decides to upgrade from a LiDAR to a 3D Camera, the developer only needs to replace the Sensor Node. As long as the new node publishes to the same topic structure, the rest of the forklift's system remains unchanged and functional.

5.2 System Communication Analysis

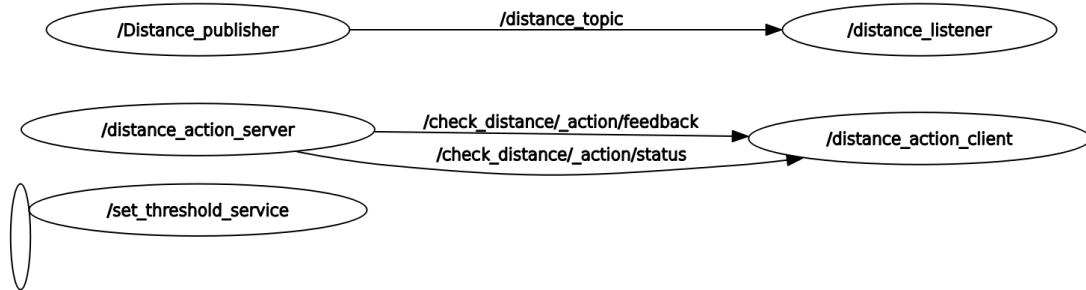


Figure 1: An example system graph showing Node connections via Topics and Actions.

Based on the visualized graph, the communication can be categorized into three distinct groups:

- **Topic-based (Pub/Sub):** The node `/Distance_publisher` acts as a data source, broadcasting messages to the `/distance_topic`. The `/distance_listener` node subscribes to this stream to receive continuous distance data for processing. This represents a one-way, asynchronous flow, ideal for high-frequency sensor data.
- **Action-based:** The graph shows connections between `/distance_action_server` and `/distance_action_client`. Unlike simple topics, this involves multiple sub-channels such as `/feedback` (for step-by-step progress reports) and `/status` (to

track the goal state). This mechanism is used for long-running tasks (the 5-step delay) where the client requires updates before receiving the final result.

- **Service-based:** Although the node `/set_threshold_service` appears isolated in the static graph, it remains in a "Listening" state. Services in ROS 2 follow a Request-Response pattern. The connection only appears momentarily when a client calls the service to update the `threshold` parameter, making it the most efficient choice for immediate, occasional configuration changes.

6 References

1. [ROS 2 Humble Documentation](#)
2. [Concepts Overview](#)
3. [Introduction to ROS and ROS2 slides](#)
4. [Beginner Tutorials \(Nodes and Topics\):](#)
5. [Kevin Wood | Robotics & AI YouTube Channel - ROS2 Tutorials](#)